PROJECT REPORT

IoT-Based Weather Station Using ESP32 and Cloud Dashboard

Internet of Things (IoT) Mini Project

Submitted by

Student Name: ASWIN S PILLAI

Date of Submission: 15-10-2026

Platforms: Wokwi Simulator | ESP32 | DHT22 | ThingSpeak

Table of Contents

Abstract 3
1. Introduction 3
2. Problem Statement 4
3. Objectives 4
4. System Architecture 4
    4.1 Function of each block 5
    4.2 Data flow 5
5. Hardware and Software Requirements 6
6. Circuit Connections 6
7. Working Principle and Methodology 7
8. ESP32 Firmware and Source Code 8
9. ThingSpeak Channel Configuration 9
10. Results and Discussion 10
    10.1 Circuit 10
    10.2 Sensor readings 11
    10.3 Wi-Fi connection and cloud transmission 12
    10.4 Dashboard monitoring 12
    10.5 High-temperature alert 13
    10.6 Summary of tests and evidence 13
11. Email Alert System and Alert History Verification 13
    11.1 Temperature Threshold and Email Alert Triggering 13
    11.2 Email delivery 15
    11.3 MATLAB alert-history verification 15
    11.4 Explanation of the alert-history code 16
    11.5 MATLAB Output for Alert History Retrieval 17
12. Limitations 17
13. Conclusion and Future Work 18
14. Submission Information 18
15. References 18
Appendix A. Complete ESP32 Source Code 20
Appendix B. Evidence Checklist 22

Abstract

This report presents the design and implementation of an IoT-based weather station that measures temperature and humidity, publishes the readings to a cloud dashboard and sends an email alert when the temperature exceeds a set limit. The system uses an ESP32 microcontroller and a DHT22 digital temperature and humidity sensor, both simulated on the Wokwi online platform. The ESP32 firmware is written in the Arduino style, reads the DHT22 on GPIO 26, validates each reading, connects to Wokwi's virtual Wi-Fi network (Wokwi-GUEST) and uploads the data to a ThingSpeak channel using an HTTP request every 15 seconds. Temperature is stored in Field 1 and humidity in Field 2, and ThingSpeak displays each field as a line chart together with a widget showing the latest value.

An email alert for the high-temperature condition was also demonstrated. A MATLAB script reads the latest temperature from Field 1 of the ThingSpeak channel and, when it exceeds the 35 °C threshold, requests an email alert through the ThingSpeak Alerts API. A received alert email reports a temperature of 51.30 °C, above that limit, and a separate MATLAB script that retrieves the alert history returned two records for the subject “Weather Station: High Temperature”, both with the status “sent”. Only the high-temperature alert is covered; no humidity alert has been implemented. All sensor values come from the Wokwi simulation, and no physical hardware was assembled.

Keywords: IoT, ESP32, DHT22, Wokwi, ThingSpeak, MATLAB, email alert, environmental monitoring, cloud dashboard.

1. Introduction

The Internet of Things (IoT) connects small embedded devices with sensors to networks so that physical quantities can be collected, stored and analysed remotely. Environmental monitoring is one of the most common IoT applications: temperature and humidity affect comfort, agriculture, storage conditions, and the operation of electronic equipment. A weather station that reports these quantities to the cloud allows the data to be viewed from any location and over any period of time instead of only at the sensor.

This mini project builds a small weather station using an ESP32 board, which has built-in Wi-Fi, and a DHT22 sensor, which provides calibrated digital temperature and humidity values. The readings are sent to ThingSpeak, an IoT analytics platform that stores incoming data in a channel and plots it automatically. ThingSpeak's alert service is then used to notify the user by email when the temperature is too high. The hardware is simulated in Wokwi, so the complete data path from sensor to cloud graph and email can be demonstrated without physical components.

2. Problem Statement

The assignment (IoT Mini Project: “IoT-Based Weather Station Using ESP32 and Cloud Dashboard”) requires the design and implementation of an IoT weather monitoring system that:

- reads temperature and humidity using a DHT22 sensor;
- sends the sensor data to the ThingSpeak cloud platform using ESP32 Wi-Fi;
- displays live graphs of temperature and humidity; and
- triggers email alerts when predefined limits are crossed.

3. Objectives

The objectives correspond to the five tasks given in the assignment:

1. **System architecture:** draw and explain the block diagram showing the DHT22, ESP32, Wi-Fi and the ThingSpeak cloud (Question 1).
2. **Sensor interfacing:** write firmware that interfaces the DHT22 with the ESP32 and acquires temperature and humidity data (Question 2).
3. **Cloud data upload:** transmit the data from the ESP32 to ThingSpeak and verify that the data fields update (Question 3).
4. **Data visualization:** configure ThingSpeak to display graphs for temperature and humidity (Question 4).
5. **Alert mechanism:** implement an email alert that triggers when a set threshold is crossed (Question 5). *Status: demonstrated for the high-temperature condition (threshold 35 °C); see Section 11. No humidity alert has been implemented.*

4. System Architecture

Figure 1 shows the block diagram of the system. Data flows from the sensor on the left through the cloud to the graphs; the alert path leads from the ThingSpeak cloud to the user's email inbox.

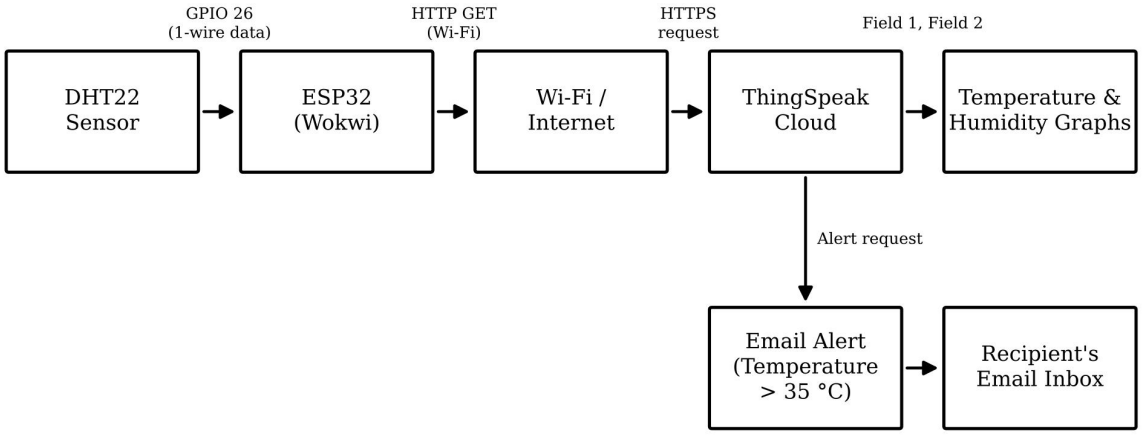


Figure 1. Block diagram of the IoT weather station.

4.1 Function of each block

- **DHT22 sensor:** measures ambient temperature and relative humidity and outputs them as a digital signal on a single data line.
- **ESP32:** the microcontroller that reads the DHT22, checks that the reading is valid, formats the HTTP request and transmits it. Its built-in Wi-Fi radio provides network access.
- **Wi-Fi / Internet:** carries the request from the ESP32 to ThingSpeak. In the simulation the ESP32 joins Wokwi's virtual access point “Wokwi-GUEST”.
- **ThingSpeak cloud:** receives the request, validates the Write API Key, stores the values in Field 1 (temperature) and Field 2 (humidity) with a timestamp, and makes them available to charts and widgets.
- **Graphs:** ThingSpeak line charts that plot each field against time, with widgets for the latest values.
- **Email alert:** a MATLAB script reads the latest Field 1 temperature and, if it exceeds the 35 °C threshold, requests an email alert through the ThingSpeak Alerts API (Section 11).

4.2 Data flow

The ESP32 periodically reads the sensor, builds a URL containing the Write API Key and the two field values, and sends it to api.thingspeak.com over the Wi-Fi connection. ThingSpeak stores the entry and the channel charts refresh to show the new point. A MATLAB script then reads the latest temperature from ThingSpeak and, if it exceeds the configured threshold, requests an alert through the ThingSpeak Alerts API, and an email is delivered to the user (Section 11).

5. Hardware and Software Requirements

The ESP32 and DHT22 used in this project are simulated in Wokwi. No physical hardware was assembled.

Item	Description / Use
ESP32 development board (simulated)	Microcontroller with built-in Wi-Fi; runs the firmware.
DHT22 sensor (simulated)	Digital temperature and humidity sensor.
Wokwi simulation platform	Online simulator for the ESP32 circuit, virtual Wi-Fi and Serial Monitor.
Arduino-style ESP32 firmware (sketch.ino)	C++ program written in the Arduino framework.
ThingSpeak cloud platform	Receives data, stores it in channel fields, plots graphs and provides the alert service.
MATLAB	Runs the script that checks the temperature and requests the alert (thingSpeakRead, webwrite) and the script that retrieves the alert history (webread).
WiFi.h	ESP32 Arduino core library for Wi-Fi connection.
HTTPClient.h	ESP32 Arduino core library for sending HTTP requests.
DHT.h (DHT sensor library)	Library used to read the DHT22 (Adafruit DHT sensor library).

The assignment lists the tools as ESP32 development board, DHT22 sensor, Arduino IDE/PlatformIO and ThingSpeak. In this work Wokwi is used in place of physical hardware and a local IDE; the same sketch can be compiled in Arduino IDE or PlatformIO.

6. Circuit Connections

The DHT22 is connected to the ESP32 as listed in Table 1 and shown in Figure 2.

DHT22 pin	ESP32 pin	Purpose
VCC	3V3	Supplies 3.3 V power to the sensor
DATA	GPIO 26	Carries the digital measurement data
NC	Not connected	Pin unused
GND	GND	Common ground reference

Table 1. DHT22 to ESP32 connections.

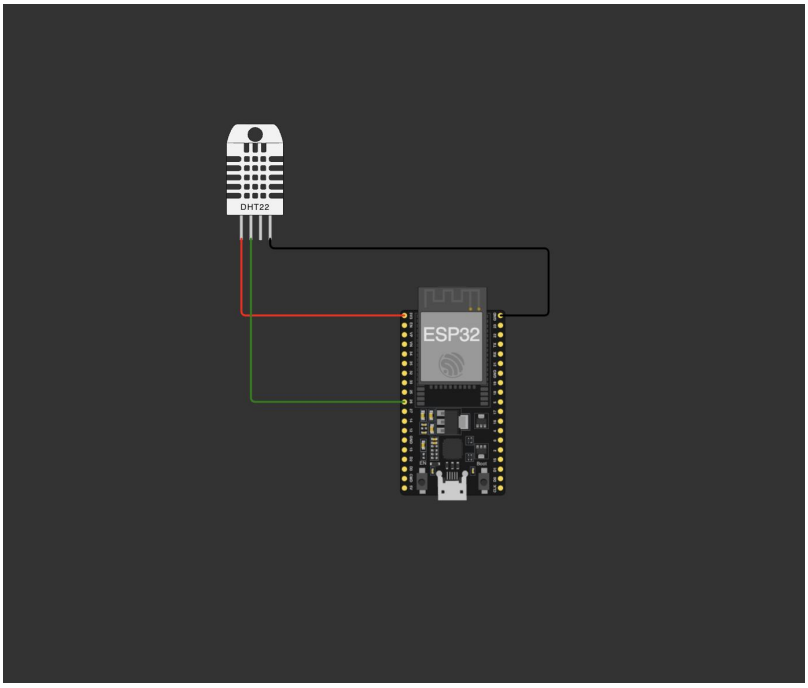


Figure 2. ESP32 and DHT22 Sensor Circuit in Wokwi (red: VCC to 3V3, green: DATA to GPIO 26, black: GND to GND).

Figure 2 shows the ESP32 and DHT22 arrangement used in the Wokwi simulation. The red wire supplies power, the black wire completes the ground return and the green wire is the single data line read by the firmware through the pin definition DHTPIN 26. The DHT22 uses a proprietary single-wire protocol in which the ESP32 sends a start signal and the sensor replies with a data frame containing humidity and temperature.

7. Working Principle and Methodology

1. Initialize the DHT22: the sketch creates a DHT object for GPIO 26 and type DHT22, and calls dht.begin() in setup().
2. Connect to Wi-Fi: WiFi.begin() joins the Wokwi-GUEST network (open network, empty password, channel 6). The program waits (checking every 500 ms), printing dots, until the status is WL_CONNECTED and then prints “Wi-Fi Connected!”.
3. Read temperature and humidity: in loop(), dht.readTemperature() returns degrees Celsius and dht.readHumidity() returns percent relative humidity.
4. Validate the readings: if either value is NaN (not a number) the read failed; the program reports the error, waits 2 seconds and skips the upload so that invalid data is never sent.
5. Prepare the cloud request: a URL is assembled from the ThingSpeak update address, the Write API Key, and the values for field1 and field2.
6. Check Wi-Fi: before uploading, the program checks that WiFi.status() is still WL_CONNECTED. If it is not, it prints “Wi-Fi disconnected. Upload skipped.” and sends nothing.

7. Send to ThingSpeak: an HTTP GET request is sent with HTTPClient.
8. Check the response: the HTTP status code is printed first; if it is positive the ThingSpeak response body is read and printed, and a body greater than 0 (the entry ID) means a new entry was stored.
9. Update Field 1 and Field 2: ThingSpeak stores temperature in Field 1 and humidity in Field 2 as one new channel entry.
10. Display in graphs: the channel's Field 1 and Field 2 charts plot the stored values against time, and widgets show the latest values.
11. Alert on high temperature: a MATLAB script reads the latest Field 1 temperature; if it exceeds 35 °C, it requests an email alert through the ThingSpeak Alerts API (Section 11).
12. Wait 15 seconds and repeat.

8. ESP32 Firmware and Source Code

The complete source code is given in Appendix A and supplied separately as IoT_Weather_Station_Firmware_Final.ino. The Write API Key is shown as a placeholder (YOUR_WRITE_API_KEY) and must be replaced by the user's own key before running. The important parts are explained below.

- **Libraries:** WiFi.h provides the Wi-Fi interface, HTTPClient.h sends HTTP requests, and DHT.h reads the DHT22.
- **Pin definitions:** #define DHTPIN 26 selects GPIO 26 for the data line and #define DHTTYPE DHT22 selects the sensor model. DHT dht(DHTPIN, DHTTYPE) creates the sensor object.
- **Wi-Fi settings:** ssid is “Wokwi-GUEST” with an empty password, as required by the Wokwi virtual network; the extra argument 6 in WiFi.begin() is the Wi-Fi channel used by the Wokwi network.
- **Sensor reading and validation:** isnan() detects a failed read; the program then prints “Failed to read DHT22”, waits 2 seconds and returns to the start of loop() without uploading.
- **Wi-Fi check:** before each upload the program tests WiFi.status() == WL_CONNECTED. If the connection has been lost, it prints “Wi-Fi disconnected. Upload skipped.” instead of skipping silently, and the loop continues with the next reading after 15 seconds.
- **HTTP request:** the URL https://api.thingspeak.com/update?api_key=<key>&field1=<temperature>&field2=<humidity> is sent with http.GET().
- **Response checking:** the HTTP status returned by http.GET() is stored in httpResponseCode and printed first as “ThingSpeak HTTP status: <code>”. A

negative value means the request itself failed (no reply from the server); the sketch then prints “ThingSpeak HTTP request failed.” followed by the reason from http.errorToString(). A positive value means the server replied, so the ThingSpeak response body is read with http.getString() and printed as “ThingSpeak response body: <text>”. If the status is 200 (HTTP_CODE_OK) the body is checked: a positive number is the ID of the new channel entry and the sketch prints “ThingSpeak entry stored successfully.”, while 0 means no new entry was created and it prints “No new ThingSpeak entry was created.” Any other positive status prints “ThingSpeak returned an HTTP error.” together with the body. http.end() is always called after the request.

- **Status code versus response body:** they carry different information. HTTP status 200 indicates a successful HTTP response, i.e. the server received and answered the request. The response body is ThingSpeak's own answer: a positive entry ID if the data was stored, or 0 if no entry was created (for example, if updates are sent faster than the 15-second limit). The body must therefore also be checked to confirm that a new entry was stored.
- **Upload interval:** delay(15000) makes one upload every 15 seconds, matching the minimum update interval of a free ThingSpeak channel.

Testing and verification: the corrected firmware in Appendix A was tested in the Wokwi simulation, and the required Serial Monitor evidence was captured. The evidence shown in this report is the DHT22 readings and the HTTP status 200 printed in the Serial Monitor (Figures 4 and 5), the ESP32 communicating with ThingSpeak over the virtual Wi-Fi network, and the resulting temperature and humidity data on the ThingSpeak dashboard (Figure 3). The firmware prints the HTTP status and the ThingSpeak response body separately: the status shows that the server answered, and the body (a positive entry ID or 0) shows whether a new entry was stored. If Wi-Fi is lost or a request fails, the firmware reports it in the Serial Monitor (“Wi-Fi disconnected. Upload skipped.” or “ThingSpeak HTTP request failed.”). All testing was carried out in simulation; no physical hardware was used.

9. ThingSpeak Channel Configuration

A ThingSpeak channel with two fields is used:

Field	Quantity	Dashboard items
Field 1	Temperature (°C)	Temperature line chart and Temperature widget
Field 2	Humidity (%)	Humidity line chart and Humidity widget

Table 2. Channel field configuration.

The channel is public and its channel ID is 3532891. The ESP32 selects the fields through the URL parameters field1 and field2 of the update request, so each upload writes one temperature value and one humidity value with a common timestamp. The Write API Key in

the request authorizes the write. ThingSpeak then draws each field as a line chart with time on the x-axis, and the widgets show the most recent value of each field. The dashboard is shown in Figure 3.

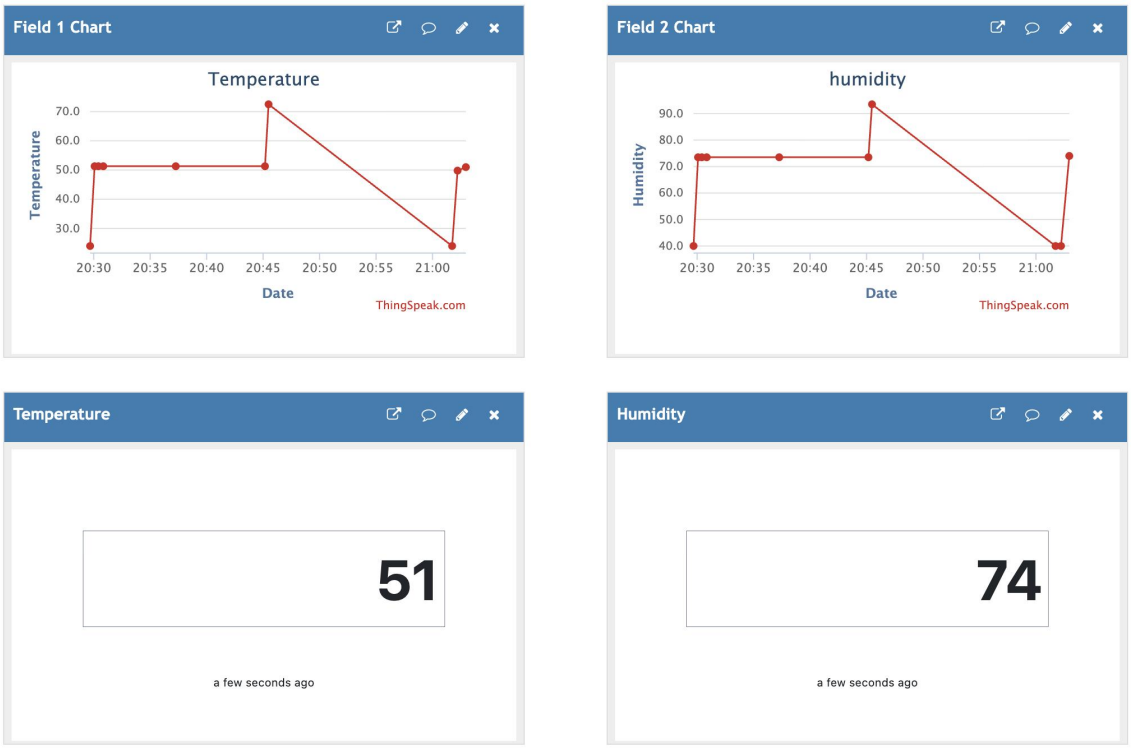


Figure 3. ThingSpeak Dashboard Showing Temperature and Humidity Trends.

The upper charts show the recorded readings over time, and the lower widgets display the latest reported values, approximately 51 °C and 74 %. The data come from the Wokwi simulation, so the graphs do not represent continuous physical-world measurements.

10. Results and Discussion

This section discusses the functions demonstrated by the supplied screenshots. All sensor values are simulated in Wokwi and are not measurements from physical hardware.

10.1 Circuit

Figure 2 shows the Wokwi circuit with the DHT22 connected to 3V3, GPIO 26 and GND, matching Table 1.

10.2 Sensor readings

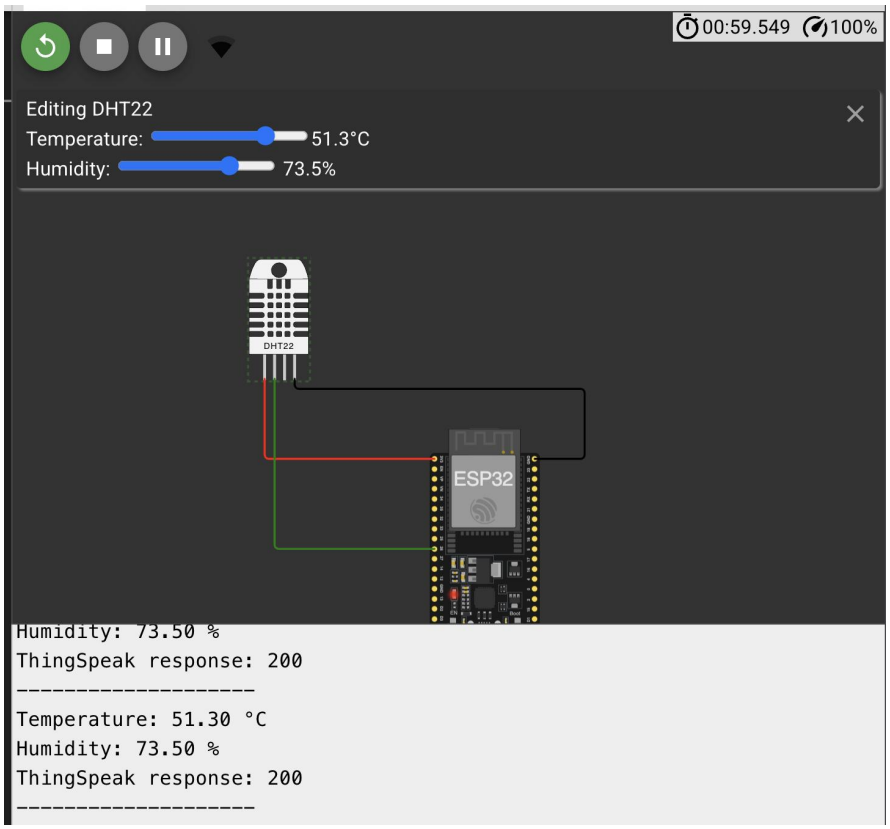


Figure 4. Sensor Readings and Cloud Communication Status.

In Figure 4 the DHT22 properties panel is set to 51.3 °C and 73.5 %, and the Serial Monitor prints Temperature: 51.30 °C and Humidity: 73.50 %, each followed by “ThingSpeak response: 200”. The values are set with the simulation sliders. The match between the slider values and the printed values shows that the firmware reads the DHT22 correctly in the simulation, and the status 200 shows that the HTTP request to ThingSpeak was answered successfully.

10.3 Wi-Fi connection and cloud transmission

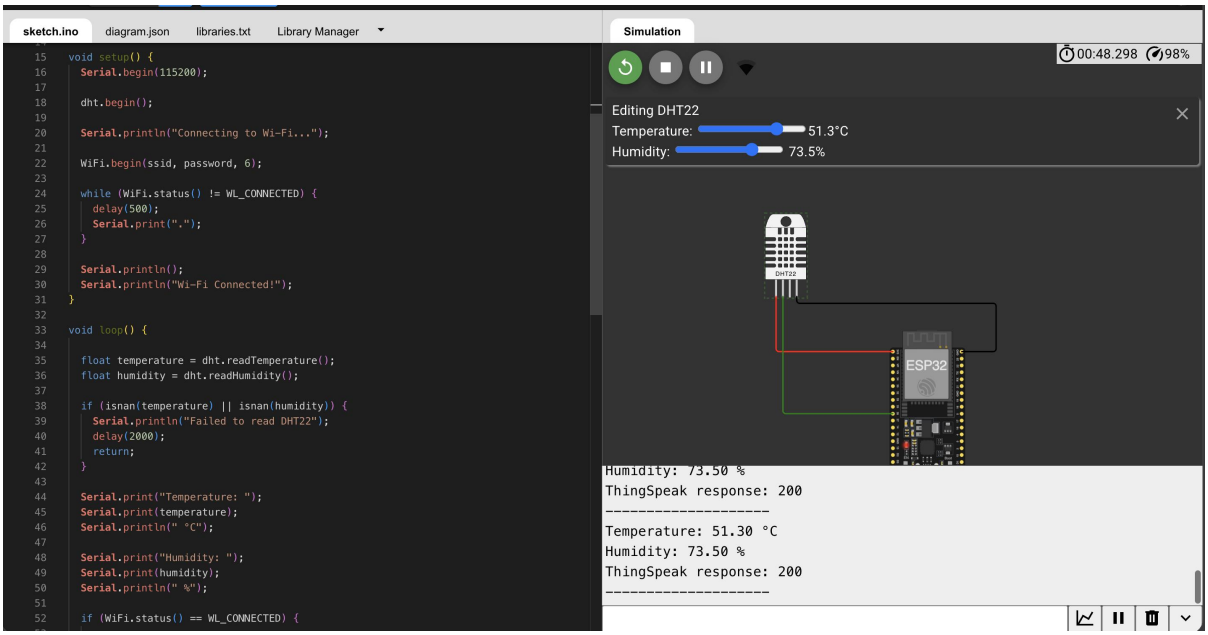


Figure 5. Wi-Fi Connection and ThingSpeak Data Transmission.

Figure 5 shows the sketch (lines 15 to 52 are visible), the simulation with the DHT22 set to 51.3 °C and 73.5 %, and the Serial Monitor with repeated readings, each followed by "ThingSpeak response: 200" and a separator line. This provides evidence of the simulated system's communication process. The connection messages printed at start-up have scrolled out of view in this screenshot; however, the sketch uploads data only when WiFi.status() equals WL_CONNECTED, so the HTTP responses indicate that the ESP32 had joined the virtual network. As explained in Section 8, the status code alone does not prove that an entry was stored; the dashboard in Figure 3 provides additional evidence that temperature and humidity data reached the channel.

10.4 Dashboard monitoring

Figure 3 shows the Temperature and Humidity charts and the latest-value widgets. On the Temperature chart most points lie near 51 °C, one point is near 72 °C around 20:45, one is near 24 °C shortly after 21:00, and the latest points are near 50 to 51 °C. The Humidity chart follows the same pattern: most points are near 73 %, one is near 93 % around 20:45, one is near 40 % shortly after 21:00, and the latest point is near 74 %. These values are consistent with the simulated inputs of Figure 4 (51.3 °C and 73.5 %) and are different settings of the simulation sliders over time, not real weather. The straight lines joining widely separated points show the change between recorded readings and do not represent continuous measurement.

10.5 High-temperature alert

The alert test and its evidence are described in Section 11: the received email (Figure 6) reports 51.30 °C against the 35 °C threshold, and the MATLAB output (Section 11.5) lists two alert records with the status “sent”.

10.6 Summary of tests and evidence

Test	Observed result	Evidence
Circuit	DHT22 connected to 3V3, GPIO 26 and GND	Figure 2
Sensor monitoring	Temperature and humidity readings are displayed in the Serial Monitor	Figure 4
Wi-Fi and cloud communication	HTTP response 200 is displayed during transmission	Figures 4 and 5
Dashboard monitoring	Temperature and Humidity trends and latest values are displayed	Figure 3
High-temperature condition	The reported temperature of 51.30 °C exceeds the 35 °C threshold; alert email received	Figure 6
Alert-history retrieval	Two records are returned, both with status sent	MATLAB output (Section 11.5)

Table 3. Summary of tests and evidence.

11. Email Alert System and Alert History Verification

This section documents the email alert required by Question 5. It covers the MATLAB code that checks the temperature and requests the alert, the received email, and a separate MATLAB script that retrieves the alert history. Only the high-temperature alert is covered.

11.1 Temperature Threshold and Email Alert Triggering

The configured high-temperature threshold is 35 °C. The alert workflow is as follows:

1. The ESP32 reads temperature and humidity using the DHT22 sensor.
2. The ESP32 uploads the sensor readings to ThingSpeak.
3. MATLAB reads the latest temperature from Field 1 of the public ThingSpeak channel 3532891; no Read API key is needed.
4. If a valid temperature exceeds 35 °C, MATLAB sends an email alert using the ThingSpeak Alerts API.
5. The email contains the measured temperature, the 35 °C threshold and a warning message.
6. If no valid reading is available, or the temperature is 35 °C or lower, this code does not send an alert.

In the test, the simulated DHT22 temperature was 51.3 °C (Figures 4 and 5), and the received alert email (Figure 6) reports 51.30 °C, which exceeds the 35 °C threshold. Listing 1 shows

the MATLAB code, which is also supplied as the separate file Alert_Trigger.m. The Alerts API key is a placeholder (YOUR_ALERTS_API_KEY) and must be replaced by the user's own key, which should be kept private.

```
channelID = 3532891;
alertApiKey = 'YOUR_ALERTS_API_KEY';

temperature = thingSpeakRead(channelID, ...
    'Fields', 1, ...
    'NumPoints', 1);

if isempty(temperature) || isnan(temperature)
    disp("No temperature reading available.");

elseif temperature > 35
    alertUrl = "https://api.thingspeak.com/alerts/send";

    options = weboptions("HeaderFields", ...
        ["ThingSpeak-Alerts-API-Key", alertApiKey]);

    alertSubject = "Weather Station: High Temperature";

    alertBody = sprintf( ...
        "Warning! Temperature is %.2f C, above the 35 C limit.", ...
        temperature);

    webwrite(alertUrl, "subject", alertSubject, ...
        "body", alertBody, options);

    disp("Temperature alert sent.");

else
    disp("Temperature is within the limit.");
end
```

Listing 1. MATLAB code for the temperature threshold check and email alert triggering.

Explanation of the alert-triggering code

- **thingSpeakRead:** reads data from a ThingSpeak channel. With channelID 3532891, 'Fields' set to 1 and 'NumPoints' set to 1, it returns the latest value of Field 1, the temperature. Because the channel is public, no Read API key is used.
- **Reading check and threshold:** the code distinguishes three cases. If no valid reading is available (isempty(temperature) || isnan(temperature)), it prints “No temperature reading available.” and sends nothing. If temperature > 35, it requests the email alert. Otherwise, for a valid reading of 35 °C or lower, it prints “Temperature is within the limit.”
- **Alert API endpoint:** alertUrl, https://api.thingspeak.com/alerts/send, is the ThingSpeak Alerts API endpoint that sends the email.
- **API key header:** weboptions places the Alerts API key in the request header ThingSpeak-Alerts-API-Key, which authorizes the request.

- **Subject and message:** alertSubject is “Weather Station: High Temperature”, and sprintf builds alertBody, which contains the measured temperature to two decimal places and the 35 C limit.
- **webwrite:** webwrite(alertUrl, "subject", alertSubject, "body", alertBody, options) sends the request to the endpoint with the subject and body, after which ThingSpeak delivers the email. The script then prints “Temperature alert sent.”

The script checks the temperature once each time it is executed. For ongoing monitoring it must be executed or scheduled to run repeatedly; it does not run continuously by itself.

11.2 Email delivery

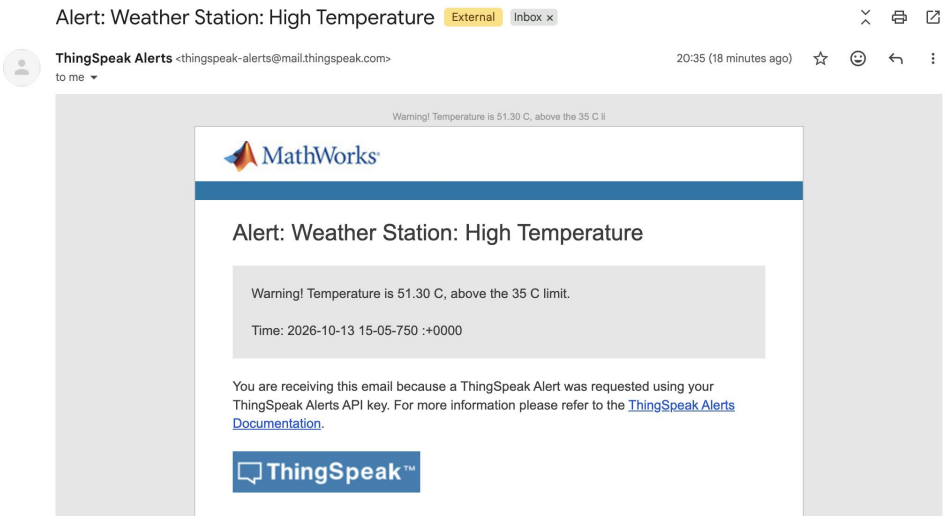


Figure 6. Received Email Notification for High Temperature.

Figure 6 shows the alert email in the recipient's inbox. The sender is “ThingSpeak Alerts”, the subject is “Alert: Weather Station: High Temperature”, and the message reads “Warning! Temperature is 51.30 C, above the 35 C limit.” The wording matches the alertSubject and alertBody of Listing 1, with ThingSpeak's “Alert: ” prefix on the subject. The screenshot therefore demonstrates receipt of the high-temperature email notification. The time in the message body (2026-10-13 15-05-750, offset +0000) appears consistent with the earlier request time recorded in the alert history (15:05:07.750 UTC, Section 11.5). The screenshot shows one received email; the second alert record listed in the history is not shown in a screenshot. Email receipt, shown here, is independent of the alert-history retrieval described next, which only reads existing records.

11.3 MATLAB alert-history verification

Alert-history retrieval is a separate operation from alert triggering (Table 4). MATLAB can retrieve existing records from the history endpoint of the ThingSpeak Alerts API. The program in Listing 2 displays the request time, send time, subject and status of each record. It is also supplied as the separate file Alert_History_Verification.m. The key is a placeholder (YOUR_ALERTS_API_KEY) and must be replaced by the user's own Alerts API key.

Script	Purpose	Sends an email?	Output
Alert triggering (Listing 1)	Reads the latest Field 1 temperature and requests an alert if it exceeds 35 °C	Yes, through the Alerts API	Prints that an alert was sent, or that the temperature is within the limit
Alert history (Listing 2)	Retrieves previous alert records	No	Requested time, sent time, subject and status

Table 4. Alert triggering compared with alert-history retrieval.

```
apiKey = "YOUR_ALERTS_API_KEY";

url = "https://api.thingspeak.com/alerts/history";

options = weboptions( ...
    "HeaderFields", ...
    ["ThingSpeak-Alerts-API-Key", apiKey]);

try
    result = webread(url, options);
    disp(result);

    history = struct2table(result, "AsArray", true);
    disp(history(:, {'requestedAt', 'sentAt', 'subject', 'status'}));

catch ME
    fprintf("Request failed: %s\n", ME.message);
end
```

Listing 2. MATLAB code for alert-history retrieval.

This code retrieves and displays existing alert-history records. It does not itself trigger or send the email notification.

11.4 Explanation of the alert-history code

- **apiKey:** stores the Alerts API key used to authorize the request.
- **url:** specifies the ThingSpeak Alerts API history endpoint, https://api.thingspeak.com/alerts/history.
- **weboptions:** configures the HTTP request header using the header name ThingSpeak-Alerts-API-Key.
- **webread:** sends the request and retrieves the alert-history response.
- **disp(result):** displays the raw returned records.
- **struct2table:** converts the returned structure array into a MATLAB table.
- **history(:, {'requestedAt', 'sentAt', 'subject', 'status'}):** selects the four columns required for verification.
- **catch ME:** displays an error message if the request or the subsequent processing fails. The table conversion is placed inside the try block so that the program does not process result after a failed request.

11.5 MATLAB Output for Alert History Retrieval

The following output shows the results of the MATLAB alert-history verification script, which returned two alert records with status sent.

```
2×1 struct array with fields:

    requestedAt
    sentAt
    subject
    status

        requestedAt                 sentAt                  subject                                  status
    ____________________________    ____________________________    ____________________________________    _______

    {'2026-10-13T15:06:31.200Z'}    {'2026-10-13T15:06:31.281Z'}    {'Weather Station: High Temperature'}    {'sent'}
    {'2026-10-13T15:05:07.750Z'}    {'2026-10-13T15:05:08.023Z'}    {'Weather Station: High Temperature'}    {'sent'}
```

Listing 3. MATLAB output for the alert-history request.

The same records are shown in Table 5.

Requested time (UTC)	Sent time (UTC)	Alert subject	Status
2026-10-13 15:06:31.200	2026-10-13 15:06:31.281	Weather Station: High Temperature	sent
2026-10-13 15:05:07.750	2026-10-13 15:05:08.023	Weather Station: High Temperature	sent

Table 5. Alert-history records returned by MATLAB.

Interpretation

- The API returned two alert-history records.
- Both records have the subject “Weather Station: High Temperature”.
- Both records have the status “sent”.
- The timestamps ending in Z are in UTC.
- The records demonstrate that the history request retrieved two alerts marked as sent.

The request and send timestamps are close together, indicating that the recorded sending events occurred shortly after their corresponding requests. The history confirms the recorded status of the two alerts. The status “sent” alone is not proof of inbox delivery; the received-email screenshot (Figure 6) independently demonstrates receipt of the alert shown in it.

12. Limitations

- The ESP32 and DHT22 are simulated in Wokwi; no physical hardware was assembled, and the sensor values come from simulation sliders rather than a real environment.
- Only the high-temperature alert (threshold 35 °C) has been demonstrated. No humidity alert or other alert condition has been implemented.
- The alert script checks the latest Field 1 value only when it is executed. It must be run or scheduled repeatedly and does not monitor continuously by itself.

- The history script only retrieves existing alert records; it does not send alerts. The script was tested successfully and retrieved two alert-history records with status sent. The received email is shown in Figure 6.
- The alert-history status “sent” records the outcome of the alert request; it is not proof that an email reached an inbox. Inbox receipt is shown only by the one email in Figure 6.
- The evidence supplied covers only a short period of simulated operation; long-term operation has not been demonstrated.

13. Conclusion and Future Work

An ESP32-based weather station was built and simulated in Wokwi. The DHT22 sensor is read on GPIO 26, the ESP32 connects to the virtual Wi-Fi network, the readings are sent to ThingSpeak (Field 1 temperature, Field 2 humidity) and ThingSpeak displays them as charts with latest-value widgets. A MATLAB script reads the latest Field 1 temperature and requests an email alert through the ThingSpeak Alerts API when it exceeds 35 °C. A received email reports 51.30 °C, and a separate MATLAB script retrieved two alert-history records with the status “sent”. These results cover Tasks 1 to 5 of the assignment, with Task 5 demonstrated for the high-temperature condition only.

Future work includes adding a humidity alert and further thresholds, scheduling the alert script so that the temperature is checked repeatedly, improving reliability (for example, retrying a failed upload), extending the testing over a longer period, and building the circuit with physical components.

14. Submission Information

As per the submission guidelines, the Project Report (PDF), the source code (.ino/.cpp) and screenshots of the ThingSpeak dashboard and alerts are to be uploaded to a Google Drive folder named IoT_Weather_Station_<Aswin_S_Pillai>, and the shared link submitted. The dashboard screenshot is Figure 3 and the alert screenshot is Figure 6 of this report.

15. References

1. Wokwi, “Wokwi Documentation (ESP32 and DHT22 simulation),” https://docs.wokwi.com/.
2. MathWorks, “ThingSpeak Documentation,” https://www.mathworks.com/help/thingspeak/.
3. MathWorks, “ThingSpeak Alerts API,” https://www.mathworks.com/help/thingspeak/thingSpeakAlerts.html.
4. MathWorks, “thingSpeakRead,” MATLAB documentation, https://www.mathworks.com/help/thingspeak/thingspeakread.html.

5. MathWorks, “weboptions,” MATLAB documentation, https://www.mathworks.com/help/matlab/ref/weboptions.html.
6. MathWorks, “webwrite,” MATLAB documentation, https://www.mathworks.com/help/matlab/ref/webwrite.html.
7. MathWorks, MATLAB documentation for webread and struct2table.
8. Adafruit, “DHT sensor library,” GitHub, https://github.com/adafruit/DHT-sensor-library.
9. Espressif Systems, “Arduino-ESP32 Documentation” (WiFi and HTTPClient libraries), https://docs.espressif.com/projects/arduino-esp32/en/latest/.

Appendix A. Complete ESP32 Source Code

File: IoT_Weather_Station_Firmware_Final.ino (API key shown as placeholder). This is the corrected firmware with Wi-Fi-disconnection handling and response-body checking, tested in the Wokwi simulation (see Section 8).

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "DHT.h"

#define DHTPIN 26
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "Wokwi-GUEST";
const char* password = "";

const char* writeAPIKey = "YOUR_WRITE_API_KEY";

void setup() {
  Serial.begin(115200);

  dht.begin();

  Serial.println("Connecting to Wi-Fi...");

  WiFi.begin(ssid, password, 6);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi Connected!");
}

void loop() {

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read DHT22");
    delay(2000);
    return;
  }

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
```

```
    String url = "https://api.thingspeak.com/update?api_key=";
    url += writeAPIKey;
    url += "&field1=";
    url += String(temperature);
    url += "&field2=";
    url += String(humidity);

    http.begin(url);
    int httpResponseCode = http.GET();

    // Check the HTTP status code first
    Serial.print("ThingSpeak HTTP status: ");
    Serial.println(httpResponseCode);

    if (httpResponseCode > 0) {
      // A positive status means the server replied, so read the body
      String responseBody = http.getString();

      Serial.print("ThingSpeak response body: ");
      Serial.println(responseBody);

      if (httpResponseCode == HTTP_CODE_OK) {
        // HTTP 200 = server answered; the body shows if an entry was stored
        if (responseBody.toInt() > 0) {
          Serial.println("ThingSpeak entry stored successfully.");
        } else {
          Serial.println("No new ThingSpeak entry was created.");
        }
      } else {
        Serial.println("ThingSpeak returned an HTTP error.");
      }
    } else {
      Serial.println("ThingSpeak HTTP request failed.");
      Serial.println(http.errorToString(httpResponseCode));
    }

    http.end();
  } else {
    Serial.println("Wi-Fi disconnected. Upload skipped.");
  }

  Serial.println("--------------------");

  delay(15000);
}
```

Appendix B. Evidence Checklist

Item	Status	Where
Wokwi circuit screenshot	Included	Figure 2
Serial Monitor readings	Included	Figure 4
Wi-Fi and cloud transmission screenshot	Included	Figure 5
ThingSpeak dashboard (charts and widgets)	Included	Figure 3
Received high-temperature alert email	Included	Figure 6
MATLAB alert-history code and output	Included (text)	Section 11.3, 11.5
MATLAB alert-triggering code	Included (text)	Section 11.1